

-  [PeliHub — Informe del proyecto](#)
 - [1. ¿Para qué sirve?](#)
 - [2. Tecnologías](#)
 - [3. Estructura del proyecto](#)
 - [4. El recorrido de una petición \(de la URL a la pantalla\)](#)
 - [5. Los modelos \(las "tablas" de datos\)](#)
 - [Película — una película del catálogo](#)
 - [EstadoPelícula — el estado de visionado de un usuario sobre una película](#)
 - [Perfil — datos extra del usuario](#)
 - [6. URLs y vistas \(tabla completa\)](#)
 - [7. La lógica clave: estado de visionado automático](#)
 - [8. Seguridad](#)
 - [9. Configuración importante \(config/settings.py\)](#)
 - [10. Cómo correr el proyecto](#)
 - [11. Tests](#)
 - [12. ★ ¿Qué pasa si modifico tal código?](#)
 - [Truco general para experimentar sin miedo](#)
 - [13. Diagramas visuales](#)
 - [13.1 Arquitectura general](#)
 - [13.2 Relación entre modelos \(entidad-relación\)](#)
 - [13.3 Máquina de estados del visionado](#)
 - [13.4 Flujo del inicio de sesión](#)
 - [14. Ejercicios prácticos](#)
 - [15. Glosario ampliado](#)



Resumen ejecutivo: PeliHub es una aplicación web hecha con **Django** que funciona como un **catálogo de películas**. Los **administradores** agregan, editan y eliminan películas (con título, director, año, género, sinopsis, puntuación y carátula), y los **usuarios normales** se registran para ver el catálogo y llevar el control de su propio visionado: cada película queda en **Pendiente**, **En progreso** o **Vista** según lo que haga el usuario, **de forma automática** (sin editar nada a mano).

► 1. ¿PARA QUÉ SIRVE?

Rol	Qué puede hacer
Visitante (sin cuenta)	Ver la landing, registrarse, iniciar sesión, recuperar contraseña.
Usuario normal	Ver el catálogo, buscar películas, abrir "Ver película" (pasa a <i>En progreso</i>), marcar "Terminar de ver" (pasa a <i>Vista</i>). No edita el estado a mano.
Administrador (superusuario)	Agregar/editar/eliminar películas, ver el panel de administración, ver cuentas registradas. No gestiona estados de visionado (eso es de cada usuario).

La idea central: **el estado de visionado se deriva del comportamiento**, no de una edición manual. Un usuario no "elige" su estado en un menú; el sistema lo deduce:

- No abrió la película → **Pendiente**
- Abrió "Ver película" pero no la terminó → **En progreso**
- Apretó "Terminar de ver" → **Vista**

► 2. TECNOLOGÍAS

Tecnología	Para qué
Python 3.13	Lenguaje del proyecto.
Django 6.1	Framework web: maneja URLs, vistas, modelos (base de datos), formularios, autenticación y plantillas.
SQLite	Base de datos (archivo <code>db.sqlite3</code>). Ideal para desarrollo: no requiere servidor.
HTML + CSS	Interfaz (plantillas en <code>templates/</code> , estilos en <code>static/peliculas/app.css</code>).
Pillow	Procesar imágenes (las carátulas).
django-axes	Anti fuerza bruta: bloquea el login tras varios intentos fallidos.
python-dotenv	Lee variables de entorno (clave secreta) desde <code>.env</code> .
WhiteNoise	Sirve archivos estáticos en producción.

► 3. ESTRUCTURA DEL PROYECTO

```

Peli_Hub/
├─ manage.py           ← comando principal (runserver, migrate, test...)
├─ db.sqlite3          ← la base de datos (no se sube a Git)
├─ .env                ← variables secretas (no se sube a Git)
├─ .env.example        ← plantilla de .env (sí se sube)
├─ requirements.txt    ← dependencias
├─ README.md           ← documentación general
├─ config/             ← configuración DEL PROYECTO
│   └─ settings.py     ← TODO lo que configura la app (BD, seguridad, apps...)
│   └─ urls.py         ← rutas "raíz" (admin, landing, películas, recuperar contrase
│   └─ wsgi.py / asgi.py ← punto de entrada para servidores
├─ mi_app/             ← app "núcleo": la página de inicio (landing)
│   └─ views.py        ← vista inicio(): redirige o muestra la landing
│   └─ urls.py         ← ruta '' → inicio
│   └─ templates/mi_app/inicio.html
└─ peliculas/          ← app PRINCIPAL (catálogo, usuarios, estados)

```

```
|— models.py          ← Pelicula, EstadoPelicula, Perfil
|— views.py           ← toda la lógica de cada página
|— urls.py            ← rutas del catálogo y autenticación
|— forms.py           ← formularios (película y registro)
|— admin.py           ← configuración del admin de Django
|— tests.py           ← pruebas automáticas (22 tests)
|— migrations/        ← historial de cambios de la base de datos
|— static/peliculas/app.css ← estilos
|— templates/peliculas/*.html ← páginas
```

Dato clave: Django separa el proyecto en *apps*. `config` es el proyecto; `peliculas` y `mi_app` son apps reutilizables. Cada app tiene su propio `models.py`, `views.py`, `urls.py` y `templates/`.

► 4. EL RECORRIDO DE UNA PETICIÓN (DE LA URL A LA PANTALLA)

Cuando escribís `http://127.0.0.1:8000/peliculas/` en el navegador:

```
Navegador → config/urls.py → peliculas/urls.py → views.lista_peliculas()
                                                    |
                                                    consulta la BD (models.Pelicula)
                                                    |
                                                    renderiza lista.html (plantilla)
                                                    |
                                                    ▼
                                                    HTML → Navegador
```

1. **URL** → el archivo `urls.py` decide qué *vista* atiende cada ruta.
2. **Vista** (`views.py`) → hace la lógica: consulta la base, valida permisos, procesa formularios.
3. **Modelo** (`models.py`) → define las tablas de la base de datos y cómo consultarlas.
4. **Plantilla** (`templates/`) → muestra los datos como HTML.

► 5. LOS MODELOS (LAS "TABLAS" DE DATOS)

Pelicula – UNA PELÍCULA DEL CATÁLOGO

Campo	Tipo	Notas
titulo	Texto (200)	Obligatorio
director	Texto (150)	Obligatorio
anio_estreno	Entero	Obligatorio
genero	Texto con opciones	accion, aventura, comedia, drama, terror, scifi, romance, documental, animacion, otros
sinopsis	Texto largo	Opcional
puntuacion	Entero (1-5)	Opcional (null=True)
portada	Imagen	Opcional, se guarda en media/portadas/
fecha_agregada	Fecha/hora	Se setea sola al crearla

`class Meta: ordering = ['-anio_estreno', '-fecha_agregada']` → el catálogo se ordena por año (más nuevo primero) y luego por fecha.

EstadoPelicula – EL ESTADO DE VISIONADO DE UN USUARIO SOBRE UNA PELÍCULA

Campo	Tipo	Notas
usuario	Clave foránea → User	A quién pertenece el estado
pelicula	Clave foránea → Pelicula	Sobre qué película
estado	Texto con opciones	pendiente , progreso , vista
actualizado	Fecha/hora	Se actualiza sola (auto_now)

`unique_together = ('usuario', 'pelicula')` → **un solo estado por usuario y película**
(no puede haber dos filas para la misma combinación).

Perfil — DATOS EXTRA DEL USUARIO

Campo	Tipo	Notas
<code>usuario</code>	Uno a uno → User	Se crea al registrarse
<code>telefono</code>	Texto (20)	Guardado en el registro

► 6. URLS Y VISTAS (TABLA COMPLETA)

Rutas de `peliculas/urls.py` (prefijo `/peliculas/`):

URL	Vista	¿Quién?	Qué hace
<code>/peliculas/</code>	<code>lista_peliculas</code>	Cualquier usuario logueado	Lista el catálogo + estado de cada película
<code>/peliculas/nueva/</code>	<code>crear_pelicula</code>	Solo admin	Formulario para agregar
<code>/peliculas/editar/<id>/</code>	<code>editar_pelicula</code>	Solo admin	Editar una película
<code>/peliculas/eliminar/<id>/</code>	<code>eliminar_pelicula</code>	Solo admin	Borrar (con confirmación)
<code>/peliculas/ver/<id>/</code>	<code>ver_pelicula</code>	Usuario normal	Marca En progreso + muestra la página de reproducción
<code>/peliculas/terminar/<id>/</code>	<code>terminar_pelicula</code>	Usuario normal (POST)	Marca Vista
<code>/peliculas/login/</code>	<code>iniciar_sesion</code>	Anónimo	Iniciar sesión

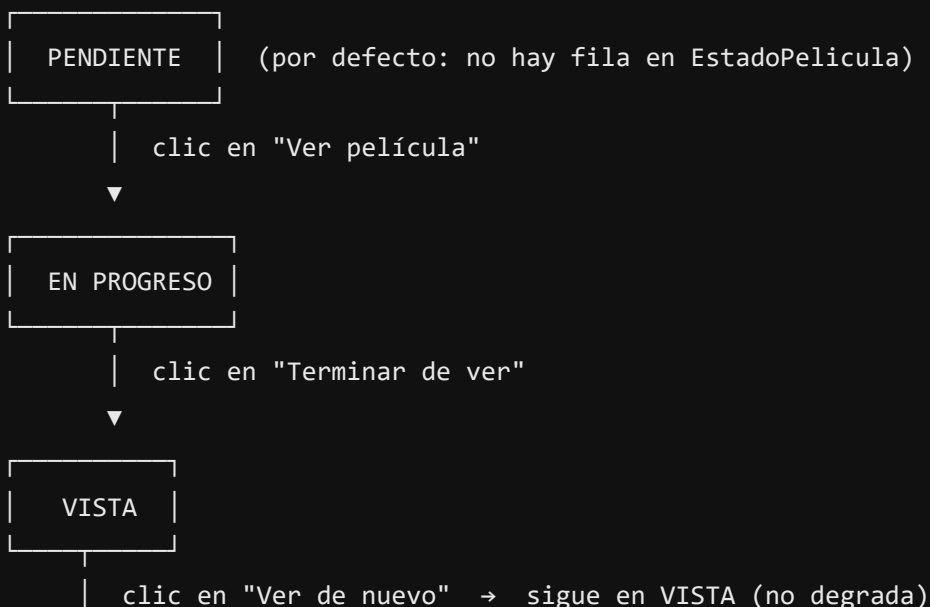
URL	Vista	¿Quién?	Qué hace
<code>/peliculas/registro/</code>	<code>registrarse</code>	Anónimo	Crear cuenta
<code>/peliculas/panel-admin/</code>	<code>panel_admin</code>	Solo admin	Panel con stats y usuarios
<code>/peliculas/logout/</code>	<code>cerrar_sesion</code>	Logueado (POST)	Cerrar sesión

Rutas de `config/urls.py` :

URL	Qué es
<code>/</code>	Landing (<code>mi_app</code> → <code>inicio</code>)
<code>/admin/</code>	Admin de Django
<code>/accounts/password-reset/ ...</code>	Recuperación de contraseña

► 7. LA LÓGICA CLAVE: ESTADO DE VISIONADO AUTOMÁTICO

Esto está en `peliculas/views.py`, funciones `ver_pelicula` y `terminar_pelicula`.



Reglas exactas (código real):

1. **Pendiente** = *no hay registro*. El sistema no crea fila hasta que el usuario interactúa. En la vista `lista_peliculas`, si no hay estado se usa `'pendiente'`.
2. **Ver película** (`ver_pelicula`): si no hay estado o el estado no es `'vista'`, se escribe `'progreso'` con `update_or_create`.
3. **Terminar** (`terminar_pelicula`): escribe `'vista'`.
4. **Ver de nuevo** una película ya `'vista'`: la condición `if not actual or actual.estado != 'vista'` evita degradarla; sigue `'vista'`.

`update_or_create(usuario=..., pelicula=..., defaults={'estado': ...})` significa: "si ya existe la fila, actualizá el estado; si no, creala". Junto con `unique_together`, garantiza una sola fila por usuario+película.

► 8. SEGURIDAD

Protección	Cómo se logra
Contraseñas y claves	<code>SECRET_KEY</code> se lee de <code>.env</code> (gitignored). Sin ella y con <code>DEBUG=False</code> , la app no arranca.
Anti fuerza bruta	<code>django-axes</code> : 5 intentos fallidos → bloqueo de 1 hora (<code>AXES_FAILURE_LIMIT</code> , <code>AXES_COOLOFF_TIME</code>).
SQL injection	El ORM de Django parametriza las consultas (no se concatenan strings SQL).
XSS	Las plantillas escapan el texto automáticamente (<code>{{ variable }}</code>).
CSRF	<code>{% csrf_token %}</code> en cada formulario POST.
Open redirect	El <code>next</code> del login se valida con <code>url_has_allowed_host_and_scheme</code> .

Protección	Cómo se logra
Logout por GET	<code>@require_POST</code> impide cerrar sesión con un simple enlace.
Permisos por rol	<code>if request.user.is_superuser</code> protege crear/editar/eliminar; <code>@login_required</code> protege las demás.
Cabeceras	<code>X-Frame-Options: DENY</code> , <code>nosniff</code> , <code>Referrer-Policy</code> , cookies <code>HttpOnly</code> / <code>SameSite</code> .
Producción	Con <code>DEBUG=False</code> : HTTPS forzado, HSTS, cookies seguras, estáticos con WhiteNoise.

► 9. CONFIGURACIÓN IMPORTANTE (`config/settings.py`)

- `DEBUG` → `True` en desarrollo, `False` en producción (cambia mucho: HTTPS, estáticos, errores detallados).
- `INSTALLED_APPS` → lista de apps activas (`mi_app` , `películas` , `axes` , etc.). **Si no está una app aquí, Django no la usa.**
- `DATABASES` → apunta a `db.sqlite3` .
- `MEDIA_URL` / `MEDIA_ROOT` → dónde se guardan y sirven las carátulas.
- `LOGIN_URL = 'login'` → a dónde redirige `@login_required` cuando no hay sesión.
- `AUTH_PASSWORD_VALIDATORS` → reglas de seguridad de contraseñas.
- `LANGUAGE_CODE = 'es'` y `TIME_ZONE = 'America/Santiago'` .

► 10. CÓMO CORRER EL PROYECTO

```
cd Peli_Hub
./venv/Scripts/python.exe manage.py migrate      # aplicar migraciones
```

```
./venv/Scripts/python.exe manage.py runserver    # levantar servidor
# abrir http://127.0.0.1:8000/
```

Otros comandos útiles:

```
./venv/Scripts/python.exe manage.py check        # verificar configuración
./venv/Scripts/python.exe manage.py test         # correr los 22 tests
./venv/Scripts/python.exe manage.py createsuperuser
./venv/Scripts/python.exe manage.py makemigrations # crear migración tras cambiar modelo
./venv/Scripts/python.exe manage.py collectstatic # juntar estáticos (producción)
```

▶ 11. TESTS

En `películas/tests.py` hay **22 pruebas** que verifican, entre otras cosas:

- El registro crea usuario + perfil.
- El catálogo muestra todas las películas, ordenadas por año.
- Un usuario normal **no puede** crear/editar/eliminar películas.
- `ver_pelicula` pone el estado en **progreso**.
- `terminar_pelicula` pone el estado en **vista**.
- El estado por defecto es **pendiente** (sin fila en la BD).
- El estado es **independiente por usuario**.
- "Ver de nuevo" no degrada una película ya vista.
- El admin no puede ver/terminar (no gestiona estados).
- El usuario normal ve el botón "Ver película"; el admin no.
- Logout exige POST (405 en GET).
- El panel admin es solo para superusuarios.

► 12. ★ ¿QUÉ PASA SI MODIFICO TAL CÓDIGO?

Esta es la parte práctica: qué efecto tiene tocar cada pieza.

#	¿Qué cambio?	¿Qué pasa?	¿Por qué?
1	Quitar <code>@login_required</code> de <code>lista_peliculas</code> (views.py)	El catálogo se vuelve público : cualquiera sin cuenta lo ve. No podrá marcar estados (esas vistas sí exigen sesión).	Ese decorador es el "portero" que exige sesión. Sin él, la vista atiende a cualquiera.
2	Quitar <code>@require_POST</code> de <code>terminar_pelicula</code>	Marcar "Vista" funcionaría con un simple enlace o recargando la página. Cambiar estado por GET es mala práctica (efectos secundarios).	El decorador obliga a que el cambio llegue por POST (más seguro, evita cambios accidentales).
3	Quitar <code>{% csrf_token %}</code> de un formulario	Al enviar, Django responde 403 "CSRF verification failed" .	Django exige ese token en cada POST para evitar falsificación de peticiones.
4	En <code>ver_pelicula</code> , cambiar la condición <code>if not actual or actual.estado != 'vista'</code> y siempre poner <code>'progreso'</code>	Una película ya Vista bajaría a En progreso al abrirla de nuevo.	Esa condición es la que "protege" el estado Vista para que no se degrade.
5	Quitar el <code>if request.user.is_superuser</code> de <code>ver_pelicula</code> / <code>terminar_pelicula</code>	El admin empezaría a tener estados de visionado propios, mezclando roles.	Ese guard separa "admin gestiona catálogo" de "usuario gestiona su visionado".
6	Agregar un género nuevo en <code>GENEROS</code> (models.py)	Hay que correr <code>makemigrations + migrate</code> ; el género nuevo aparece en el formulario y en <code>get_genero_display()</code> .	Los <code>choices</code> quedan grabados en la migración; la BD y el código deben quedar sincronizados.
7	Quitar <code>unique_together = ('usuario', 'pelicula')</code> de	Podrían existir varios estados para la misma	Esa restricción garantiza una sola fila por

#	¿Qué cambio?	¿Qué pasa?	¿Por qué?
	<code>EstadoPelicula</code>	película y usuario (filas duplicadas).	combinación.
8	Cambiar <code>ordering = ['-anio_estreno', '-fecha_agregada']</code> en <code>Pelicula.Meta</code>	Cambia el orden del catálogo (p. ej. año ascendente si sacás el <code>-</code>).	Esa meta define cómo <code>Pelicula.objects.all()</code> ordena por defecto.
9	Borrar la línea <code>urlpatterns += static(MEDIA_URL, ...)</code> en <code>config/urls.py</code> (en DEBUG)	Las carátulas dejan de cargar (404), aunque los archivos sigan en <code>media/</code> .	Esa línea es la que sirve los archivos subidos en desarrollo.
10	Cambiar <code>MEDIA_ROOT</code> a otra carpeta sin mover las imágenes	Las portadas "desaparecen" (404), porque se buscan en la carpeta nueva.	El servidor sirve las imágenes desde <code>MEDIA_ROOT</code> ; si apunta a otro lado, no las encuentra.
11	Poner <code>DEBUG = False</code> sin configurar <code>ALLOWED_HOSTS</code> ni <code>SECRET_KEY</code>	La app no arranca (falta clave) o responde 400 (host no permitido); y deja de servir estáticos/media en dev.	En producción Django endurece la seguridad: exige clave real y hosts permitidos.
12	Bajar <code>AXES_FAILURE_LIMIT = 5</code> a <code>1</code>	El login bloquea tras 1 intento fallido (en vez de 5).	Es el umbral de intentos fallidos permitidos por django-axes.
13	Poner <code>puntuacion</code> sin <code>null=True, blank=True</code>	Guardar una película sin puntuación daría error de validación (campo requerido).	<code>null=True/blank=True</code> es lo que hace el campo opcional.
14	Cambiar el texto de <code>ESTADO_DISPLAY</code> en <code>lista_peliculas</code>	Solo cambia la etiqueta visible ("Vista" → "La vi", etc.). La lógica interna no cambia.	Ese diccionario traduce el valor interno (<code>'vista'</code>) al texto que se muestra.

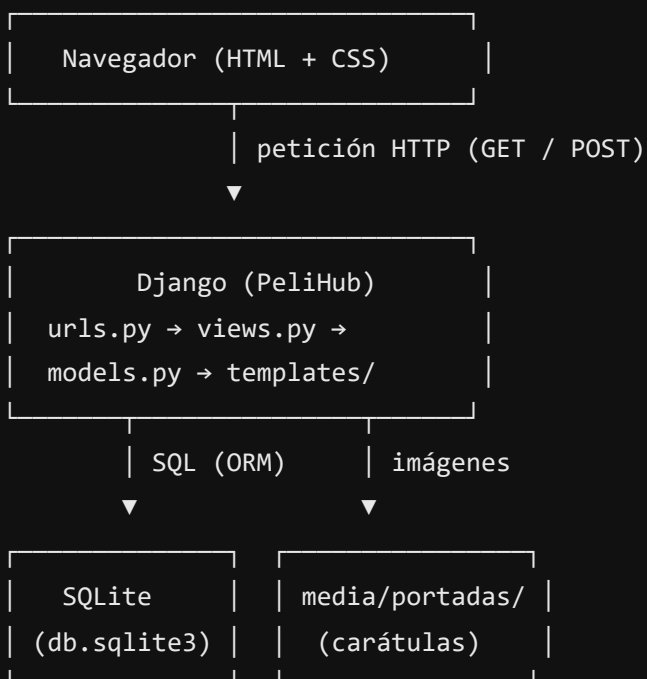
#	¿Qué cambio?	¿Qué pasa?	¿Por qué?
15	Renombrar una ruta en <code>urls.py</code> (ej. <code>ver_pelicula</code>) sin actualizar <code>{% url %}</code> / <code>reverse</code>	Error NoReverseMatch al renderizar las páginas que la usan.	Las plantillas y vistas buscan las URLs por su <i>nombre</i> ; si el nombre cambia, no lo encuentran.

TRUCO GENERAL PARA EXPERIMENTAR SIN MIEDO

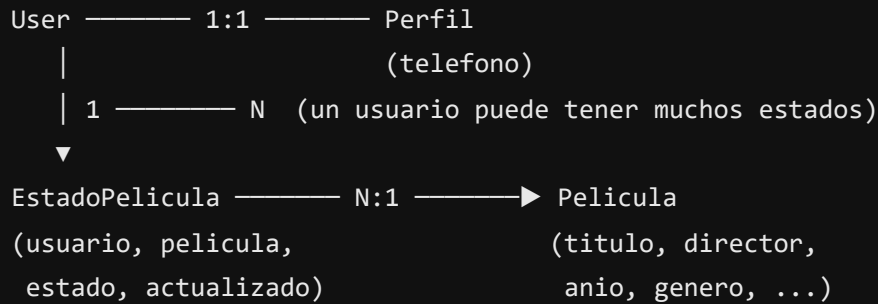
- Haz **un cambio a la vez** y mirá qué pasa.
- Usá `./venv/Scripts/python.exe manage.py check` y `manage.py test` después de cada cambio.
- Si rompés algo, `git status` te muestra qué archivos tocaste y `git restore <archivo>` lo revierte.

► 13. DIAGRAMAS VISUALES

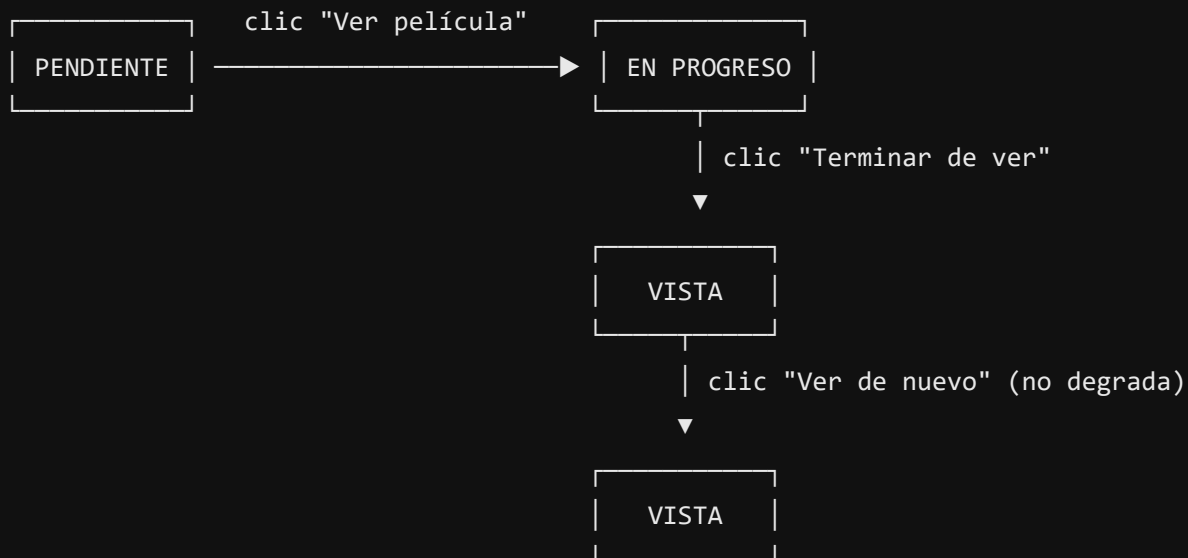
13.1 ARQUITECTURA GENERAL



13.2 RELACIÓN ENTRE MODELOS (ENTIDAD-RELACIÓN)



13.3 MÁQUINA DE ESTADOS DEL VISIONADO



13.4 FLUJO DEL INICIO DE SESIÓN

GET /peliculas/login/ —> muestra el formulario

POST usuario + contraseña —> ¿son válidos?

- sí —> login() + redirect a /peliculas/
- no —> error + django-axes suma 1 intento (a los 5 → bloqueo de 1 hora)

► 14. EJERCICIOS PRÁCTICOS

Para que cada uno pruebe con su computadora. Después de cada paso, fijate en el resultado.

1. **Registrar una cuenta nueva** (menú → "Crear cuenta"). *Qué esperarás:* todas las películas aparecen con el badge **Pendiente**.
2. **Abrir "Ver película"** de una película y **volver sin terminar**. *Qué esperarás:* queda **En progreso**.
3. **Entrar de nuevo y apretar "Terminar de ver"**. *Qué esperarás:* pasa a **Vista** y sale el mensaje de confirmación.
4. **Abrir "Ver de nuevo"** una película ya Vista. *Qué esperarás:* **sigue Vista** (no baja de categoría).
5. **Entrar como superusuario** y mirar el catálogo. *Qué esperarás:* no ves botones de estado; en su lugar ves  y .
6. **Agregar una película** (admin → "+ Agregar película"). *Qué esperarás:* aparece ordenada por año, con su carátula si subiste una.
7. **Buscar por título** en el buscador. *Qué esperarás:* la lista se filtra.
8. **Entrar a `/peliculas/nueva/` como usuario normal** (escribí la URL a mano). *Qué esperarás:* te redirige con el mensaje "Solo los administradores...".
9. **Experimento CSRF:** quitá `{% csrf_token %}` del `login.html` y enviá el formulario. *Qué esperarás:* error **403**. *Revertí el cambio después.*
10. **Experimento de orden:** en `models.py` cambiá `ordering` a `['anio_estreno']` y recargá. *Qué esperarás:* el catálogo se ordena del más viejo al más nuevo.
11. **Experimento de bloqueo:** en `settings.py` poné `AXES_FAILURE_LIMIT = 1`, fallá la contraseña una vez. *Qué esperarás:* la cuenta/IP queda bloqueada. *Revertí después.*
12. **Correr los tests:** `./venv/Scripts/python.exe manage.py test`. *Qué esperarás:* **OK** con 22 tests.

► 15. GLOSARIO AMPLIADO

Término	Qué significa
Vista (view)	Función que recibe una petición y devuelve una respuesta (HTML o redirección).
Modelo (model)	Clase de Python que define una tabla de la base de datos.
Plantilla (template)	Archivo HTML con placeholders <code>{{ variable }}</code> y <code>{% tag %}</code> .
Migración	Archivo que describe un cambio en la base de datos; <code>migrate</code> lo aplica.
ORM	Capa que traduce Python a SQL; <code>Pelicula.objects.filter(...)</code> en vez de escribir SQL a mano.
Queryset	El resultado de una consulta (<code>Pelicula.objects.all()</code>); es "perezoso", no toca la BD hasta que se usa.
Clave foránea (FK)	Relación "muchos a uno" (p. ej. cada estado pertenece a una película).
Relación 1:1	Un registro se relaciona con exactamente otro (p. ej. <code>User</code> ↔ <code>Perfil</code>).
Decorador	<code>@algo</code> encima de una función; le agrega comportamiento (exigir login, solo POST, etc.).
Petición (request)	Lo que el navegador envía al servidor; trae método (GET/POST), datos y cookies.
Respuesta (response)	Lo que el servidor devuelve (HTML, redirección, error...).
GET / POST	Métodos HTTP: GET pide datos (no cambia estado); POST envía datos y suele cambiar estado.
Redirección (redirect)	Respuesta que manda al navegador a otra URL (código 302).
Middleware	Capa que procesa cada petición/respuesta (seguridad, sesiones, CSRF...).
Sesión (session)	Memoria del servidor para recordar quién está logueado entre peticiones.
Cookie	Dato que el navegador guarda y reenvía; Django la usa para la sesión.

Término	Qué significa
Contexto (context)	Diccionario de variables que una vista pasa a la plantilla.
CSRF	Protección contra peticiones falsificadas entre sitios (token en cada formulario).
Estáticos (static)	Archivos que no cambian: CSS, JS, imágenes de diseño.
Media	Archivos subidos por los usuarios (las carátulas en <code>media/portadas/</code>).
Superusuario	Usuario administrador (<code>is_superuser=True</code>), con acceso total al admin.
Staff	Usuario que puede entrar al admin de Django pero no necesariamente es superusuario.
<code>manage.py</code>	Herramienta de línea de comandos: <code>runserver</code> , <code>migrate</code> , <code>test</code> , <code>createsuperuser</code> ...

Informe generado para explicar el proyecto PeliHub. Estado actual: 18 películas con carátula, 4 usuarios (2 admin, 2 normales), 22 tests en verde.